

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – COMPARATIVE ANALYSIS

Name	Koushik P
Reg. No.	192421178
Course	Data Structures
Course Code	CSA03
Course Outcome	CO: Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data (BL

1. Hashing vs Binary Search

Hashing gives average $O(1)$ search, while binary search gives $O(\log n)$. Hashing generally needs $O(n)$ table space; binary search needs $O(1)$ extra space on a sorted array. Hash tables support insert/delete in average $O(1)$, whereas maintaining sorted data makes updates costly. Binary search is excellent for static, ordered data; hashing is better for frequent dynamic lookup. Therefore, hashing is generally better for dynamic systems.

2. Quick Sort vs Merge Sort

Quick Sort: best/average $O(n \log n)$, worst $O(n^2)$; Merge Sort: best/average/worst $O(n \log n)$. Standard Quick Sort is not stable, while Merge Sort is stable. Quick Sort is usually in-place with $O(\log n)$ average stack space; Merge Sort normally needs $O(n)$ extra space for arrays. Quick Sort is often preferred because of good cache locality, low overhead and strong average-case performance. Randomized/pivot-balanced versions reduce worst-case risk.

3. Heap vs Sorted Array

A heap supports insertion in $O(\log n)$ and highest-priority deletion in $O(\log n)$, with $O(1)$ access to the maximum/minimum. A sorted array has $O(n)$ insertion because elements may need shifting, but deletion of the extreme element can be $O(1)$ if the end is used. For repeated insertion and priority retrieval, a heap is more efficient and is therefore preferable for scheduling systems.

4. Linear Probing vs Separate Chaining

Linear probing stores entries in the table and suffers from primary clustering. Separate chaining uses lists/buckets and avoids primary clustering but needs additional pointer/object memory. At high load factors, linear probing can degrade sharply; chaining usually remains more tolerant. For large-scale applications with unpredictable load, separate chaining is often preferred because deletion and high-load behavior are simpler.

5. Sequential vs Parallel Sorting

Sequential sorting runs on one execution stream and has little overhead. Parallel sorting can reduce execution time and scales across processors, but synchronization, thread creation and data movement add overhead. Parallel sorting is most useful for large datasets where computation dominates overhead. For small datasets, sequential sorting is often faster because parallel overhead can exceed the saved computation time.

6. Chaining vs Open Addressing for 10 Million Pairs

At load factor 0.75, separate chaining stores table entries plus bucket/link overhead, while open addressing keeps records directly in the table and can be more memory-efficient. Open addressing generally has better cache locality. Both have $O(n)$ worst-case lookup, but chaining handles clustering differently. Deletion is simpler with chaining; open addressing needs tombstones or relocation. Choose open addressing for cache-sensitive workloads and chaining when deletion and high-load flexibility matter.

7. Linear, Quadratic and Double Hashing

Linear probing has primary clustering and can have long probe sequences. Quadratic probing removes primary clustering but can still have secondary clustering for keys with the same initial hash. Double hashing gives different probe sequences and minimizes both major clustering effects. Average probe counts depend on load factor, but double hashing

usually gives the best distribution. Linear probing generally offers the best cache performance because it accesses contiguous memory.

8. Binary Heap vs Balanced BST Scheduler

A binary heap provides insert $O(\log n)$ and extract-min $O(\log n)$, while decrease-key is $O(\log n)$ when the item location is known. A balanced BST provides insert, deletion and key updates in $O(\log n)$, and supports ordered/arbitrary-key operations naturally. A heap is usually more memory-efficient because it can be stored compactly in an array. For frequent arbitrary updates and ordered searches, a balanced BST is more flexible.

9. Quick Sort vs Merge Sort for Linked Lists

For a large linked list, Merge Sort is usually preferable because splitting and merging can be done by changing links, requiring little extra node storage and $O(n \log n)$ time. Quick Sort has poor locality on linked structures and partitioning is less convenient. For arrays, Quick Sort often performs better due to cache locality and in-place operation, while Merge Sort remains attractive when stability and predictable $O(n \log n)$ time are required.

10. Heap Sort vs Quick Sort for 1 Billion Integers

Heap Sort guarantees $O(n \log n)$ worst-case time and $O(1)$ auxiliary array space, while Quick Sort has $O(n^2)$ worst case although average performance is $O(n \log n)$. Heap Sort therefore provides stronger worst-case predictability. Under strict latency requirements, predictable performance is important, so Heap Sort is safer despite Quick Sort often being faster on average.

11. Radix Sort vs Comparison Sorts

For 32-bit integers using base 256, radix sort processes a fixed number of digit passes, giving approximately $O(d \times n)$, where d is the number of byte passes. Quick/Merge Sort require $O(n \log n)$ comparisons. Radix Sort needs counting-array/buffer memory and can be very fast for fixed-width integers due to predictable operations. It loses advantage when keys are variable-length, the radix is poorly chosen, memory bandwidth is limiting, or comparison sorting has favorable cache behavior.

12. Insertion Sort vs Quick Sort for Arrays < 50

Insertion Sort has very low constant overhead, excellent cache locality and simple control flow. Quick Sort adds recursion, partitioning and function-call overhead. For very small arrays, these overheads can outweigh Quick Sort's asymptotic advantage. Therefore, production hybrid algorithms such as Timsort and Introsort switch to insertion sort for small subarrays.

13. Single-threaded Quick Sort vs Parallel Sorts

Single-threaded Quick Sort has no synchronization overhead. Parallel Quick Sort can obtain strong speedup by processing independent partitions, but load imbalance can occur when partitions are uneven. Parallel Merge Sort has regular partitioning and merging but requires synchronization and additional memory traffic. Amdahl's law limits speedup according to the serial fraction. Parallel Merge Sort often scales predictably, while well-balanced parallel Quick Sort can be faster when memory bandwidth is not saturated.

14. MapReduce vs Parallel Merge Sort for 10 TB

MapReduce sorting distributes data across nodes, uses disk-based processing and provides fault tolerance, but incurs substantial network shuffle and disk I/O overhead. Shared-memory parallel merge sort avoids distributed network communication but is limited by one machine's memory and compute resources. MapReduce becomes necessary when the dataset is too large for one machine or high fault tolerance and cluster scalability are required.

15. GPU Bitonic Sort vs CPU Parallel Quick Sort

Bitonic sort has a highly parallel structure with $O(\log^2 n)$ parallel stages, while CPU parallel Quick Sort performs about $O((n \log n)/p)$ work with p processors, ignoring overhead. GPUs provide massive parallelism but have global/shared-memory constraints, possible warp divergence and PCIe transfer costs. GPU sorting becomes advantageous when the input is sufficiently large and data can remain on the GPU, so transfer overhead is amortized.

16. Linear Probing vs Cuckoo Hashing

Linear probing has good cache locality but lookup time becomes less predictable as load increases. Cuckoo hashing provides a small, bounded number of candidate locations, giving highly predictable lookup latency. Its disadvantages are

insertion failures, possible rehashing and relocation costs. For a read-heavy workload requiring predictable latency, cuckoo hashing is often preferable.

17. Binary Heap vs Fibonacci Heap in Dijkstra

Binary Heap gives $O(\log n)$ decrease-key and $O(\log n)$ extract-min, with relatively small memory and implementation overhead. Fibonacci Heap provides amortized $O(1)$ decrease-key and $O(\log n)$ extract-min, improving theoretical bounds for Dijkstra. However, Fibonacci heaps have complex pointer-heavy structures and poor cache locality. Consequently, binary heaps are usually faster and simpler in real implementations.

18. Timsort vs Merge Sort

Timsort is adaptive: on already sorted data it can approach $O(n)$, and it also handles reverse runs efficiently. With duplicates and random data it generally performs $O(n \log n)$, with $O(n)$ worst-case auxiliary space. Traditional Merge Sort is $O(n \log n)$ in best, average and worst cases and does not exploit existing runs as aggressively. Timsort is therefore often better for real-world partially ordered data.

19. Hash Table vs BST in Multithreading

Hash tables can provide average $O(1)$ lookup and can be partitioned into independently locked buckets, enabling fine-grained concurrency. BST operations are $O(\log n)$ when balanced and require synchronization along paths. Hash-table resizing can require expensive coordination, while balanced trees avoid global table resizing. For scalable concurrent dictionary operations, a well-designed concurrent hash table generally provides higher throughput.

20. Counting vs Radix vs Bucket Sort for Uniform Floats

Counting Sort is naturally suited to discrete integer keys, so direct use with floating-point values is inappropriate unless values are quantized. Radix Sort can handle floats by using an appropriate binary/key transformation and digit passes, but requires extra buffers. Bucket Sort is especially suitable for uniformly distributed values in $[0,1)$, placing values into buckets and sorting each bucket. For 100 million uniform floats, Bucket Sort is often the most natural choice because distribution matches its assumptions.

21. Counting vs Radix vs Bucket Sort for Uniform Floats

The comparison is the same: Counting Sort requires discretization because floating values are not direct integer indices. Radix Sort can process floating-point representations after suitable key transformation and has predictable digit-pass complexity. Bucket Sort benefits strongly from uniform distribution and can approach $O(n)$ expected time with well-sized buckets. Thus, Bucket Sort is generally the best fit for uniformly distributed floats in $[0,1)$.

22. Counting vs Radix vs Bucket Sort for Uniform Floats

Counting Sort is memory-intensive and unsuitable for arbitrary non-integer values without quantization. Radix Sort is applicable after transforming float keys but requires multiple passes and auxiliary storage. Bucket Sort directly exploits the known $[0,1)$ range and uniform distribution, giving good expected performance and can be stable when each bucket uses a stable sort. Therefore, Bucket Sort is the most suitable among the three for this stated workload.

23. Quadratic Probing vs Double Hashing

Quadratic probing reduces primary clustering compared with linear probing but has secondary clustering because keys sharing the same initial hash follow the same probe pattern. Double hashing uses a second hash function, greatly reducing both primary and secondary clustering. Double hashing is generally more efficient at higher load factors, while quadratic probing can be simpler and slightly more cache-friendly.

24. Best-case vs Worst-case Complexity

Best-case complexity shows the minimum work under favorable input, while worst-case complexity gives an upper bound for the most difficult input. Worst-case analysis is especially important for systems with strict performance guarantees. Best-case behavior helps explain adaptive algorithms, but algorithm selection should usually consider average and worst-case behavior together with actual workload characteristics.

25. Recursive vs Iterative Quick Sort

Recursive Quick Sort is simple and naturally matches the divide-and-conquer algorithm, but its call stack uses space. Average stack usage is $O(\log n)$, while poor partitions can lead to $O(n)$ stack depth and possible stack overflow. An iterative implementation can manage its own stack and reduce recursion risk, but is slightly more complex. Tail-recursion

optimization or always processing the smaller partition first can reduce stack usage.

26. Merge Sort vs Radix Sort

Merge Sort is a comparison-based sorting algorithm and works with general comparable keys. Radix Sort is non-comparison-based and requires keys that can be processed digit by digit. Merge Sort is broadly applicable and has $O(n \log n)$ time. Radix Sort can approach $O(dn)$ and be faster for fixed-format integers or strings, but its input limitations make it less general.

27. Bottom-up Heap vs Repeated Insertion

Bottom-up heap construction starts from the last internal node and heapifies downward. It takes $O(n)$ time. Repeatedly inserting n elements takes $O(n \log n)$ in the worst/typical bound. Therefore, bottom-up construction is more efficient when all elements are already available because it builds the heap in linear time.

28. Heap-based vs Array-based Priority Queue

A heap-based priority queue inserts in $O(\log n)$ and deletes the highest/lowest priority item in $O(\log n)$. An unsorted array can insert in $O(1)$ but deletion of the priority item takes $O(n)$. A sorted array can delete an extreme in $O(1)$ but insertion may take $O(n)$. A heap gives the best overall balance for repeated insert and delete-priority operations.

29. Binary Heap vs BST

A binary heap is optimized for finding the minimum/maximum and does not support efficient arbitrary searching; search can be $O(n)$. A balanced BST supports search, insertion and deletion in $O(\log n)$ and maintains sorted order. Heaps are common in priority queues and scheduling, while BSTs are useful for ordered dictionaries, range queries and ordered traversal.

30. Internal vs External Sorting

Internal sorting operates when the complete dataset fits in main memory, so it can use fast RAM-based algorithms. External sorting is designed for datasets larger than available RAM and relies on secondary storage, commonly using run generation and multiway merging. External sorting is dominated by disk/SSD I/O, while internal sorting is generally faster because memory access is much quicker.

31. Parallel vs Sequential Merge Sort

Sequential Merge Sort takes $O(n \log n)$ time. Parallel Merge Sort can reduce wall-clock time by distributing independent sorting and merging tasks across processors. However, synchronization, task creation and memory-bandwidth contention create overhead. It is most beneficial for large datasets and many available cores; for small datasets, sequential execution may be faster.

32. Single-core vs Multi-core Sorting

Multi-core sorting can significantly reduce execution time by dividing independent sorting tasks among cores. However, synchronization, load balancing, shared-memory contention and thread-management overhead can reduce the theoretical speedup. Single-core sorting is simpler and predictable, while multi-core sorting is beneficial for sufficiently large workloads.

33. Time Complexity vs Space Complexity

Lower time complexity generally improves speed, while lower space complexity reduces memory requirements. The two can trade off: extra memory may enable faster hashing, buffering or divide-and-conquer operations. Memory-constrained embedded systems may prioritize space, whereas high-throughput servers may spend memory to reduce time. The choice depends on system constraints.

34. Direct Addressing vs Hashing

Direct addressing uses an array indexed directly by the key and can provide $O(1)$ lookup, but requires memory proportional to the entire key universe. Hashing maps a large or sparse key space into a smaller table and therefore uses memory more flexibly, with average $O(1)$ operations. Direct addressing is excellent for small dense integer ranges; hashing is more practical for large sparse real-world keys.

35. Load Factor vs Input Size

In hashing, increasing load factor causes more collisions and can degrade lookup and insertion performance. In sorting, increasing input size increases running time according to the algorithm's complexity, such as $O(n \log n)$. Hash tables optimize by resizing or choosing good hash functions; sorting optimizes through better algorithms, parallelism and adaptive techniques. Both show performance degradation as workload grows, but for different reasons.

36. Heap Sort vs Quick Sort

Heap Sort guarantees $O(n \log n)$ worst-case time and uses $O(1)$ auxiliary array space. Quick Sort has $O(n \log n)$ average time but $O(n^2)$ worst case, with $O(\log n)$ average recursion space. Quick Sort often has better practical performance due to cache locality and lower constants. Heap Sort is preferred when worst-case guarantees are more important.

37. Radix Sort vs Counting Sort

Counting Sort counts occurrences of each integer key and works efficiently when the key range is reasonably small. Radix Sort sorts by individual digits using a stable subroutine such as Counting Sort. Counting Sort has complexity $O(n+k)$, where k is the key range; Radix Sort is about $O(d(n+k))$ depending on the digit base. Counting Sort is ideal for small ranges, while Radix Sort is better for larger fixed-width keys.

38. Single-threaded vs Distributed Sorting

Single-threaded sorting is simple and requires one machine but has limited scalability. Distributed sorting divides data among multiple machines and can handle very large datasets, but needs networking, partitioning, coordination and fault handling. Distributed systems require substantially more resources and implementation complexity. They are justified when data volume exceeds practical single-machine capacity.

39. Binary Heap vs Fibonacci Heap

Binary Heap insertion is $O(\log n)$ in the worst case, while Fibonacci Heap insertion is $O(1)$ amortized. Binary Heap decrease-key is $O(\log n)$, whereas Fibonacci Heap provides $O(1)$ amortized decrease-key. Fibonacci Heap has better theoretical bounds but greater implementation and memory overhead and poorer cache locality. Binary Heap is therefore usually more practical.

40. Sorting + Binary Search vs Hashing

For static data, sorting once and using binary search is attractive because it gives $O(\log n)$ queries, compact storage and ordered data. For dynamic data with frequent insertions/deletions, hashing usually provides average $O(1)$ lookup and simpler updates. Thus, sorted data plus binary search is generally better for static datasets, while hashing is better for dynamic direct-lookup workloads.

Student Declaration

I, Koushik P (Reg. No. 192421178), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the student: _____